



Team Details

Team Name:

TECHNOX

SR.NO	ROLE	NAME	ACADEMIC YEAR
1	Member 1	Dharshan Raju R D	3
2	Member 2	Hitesh K P	3

 COLLEGE NAME

CHENNAI INSITUTE OF TECHNOLOGY

 TEAM LEADER CONTACT NUMBER

+91 9884293428

 TEAM LEADER EMAIL ADDRESS

Dharshanrajurd.ece2024@citchennai.net



Problem Statement Addressed



AI-Based Restoration of Degraded Images for Semiconductor Inspection | KLA, Track 1

DESCRIPTION / DETAILS

Inspection images from a semiconductor line arrive degraded three ways at once: speckle noise (multiplicative, and it pushes pixel values outside the true intensity range), additive Gaussian noise (soft, hazy edges), and 2x spatial downsampling (fine detail genuinely lost, not merely hidden). Any combination and strength may be present in a single image.

Why it matters: *these degradations hide the very features an inspection system exists to detect. Blur the noise away and real structure is destroyed; sharpen too hard and structure appears that was never there. Both failure modes translate directly into missed defects or false defect calls on a production line.*

The task: *learn one transformation from paired (degraded, clean) examples that reverses all three degradations simultaneously, generalises to structure types never seen in training, and runs fast enough to keep up with inline inspection throughput.*



Idea Description



A single compact CNN that restores all three degradations in one forward pass, and adapts its own processing to how badly each individual image is degraded.

KEY CONCEPT & APPROACH

We treat this as one joint problem rather than three sequential ones. A NAFNet-lite encoder-decoder (LayerNorm + depthwise conv + SimpleGate + simplified channel attention) does denoising, deblurring and 2x super-resolution together in a single pass, ending in a PixelShuffle upsample head. Staging them separately would let each stage's errors compound into the next; solving them jointly avoids that and is far faster at inference.

On top of that, a small degradation estimator reads each input and emits a 32-d vector describing how degraded that specific image is. A FiLM layer turns that vector into a per-channel scale and shift applied at the bottleneck, so the network adapts per image instead of applying one fixed strategy.

SOLUTION OVERVIEW

Speckle noise -> Charbonnier loss is robust to the outlier pixels speckle creates, and inputs are never clipped on load, so out-of-range values are preserved.

Gaussian blur -> a Sobel edge loss explicitly penalises smeared edges, directly targeting the 'do not blur the image to remove noise' constraint.

Downsampling -> a learned PixelShuffle head reconstructs the 2x detail rather than interpolating it. No adversarial loss is used anywhere, by design: GAN-style training invents plausible texture, which is exactly the ringing and artificial patterning the problem statement warns against.



Proposed Solution

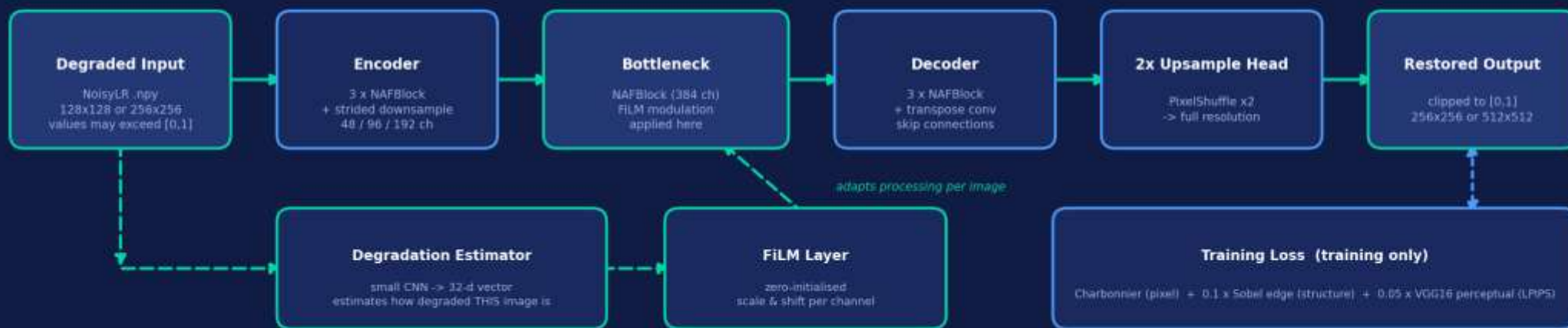
Single-pass restoration pipeline. The dashed teal path is our degradation-aware conditioning; the dashed blue path is training-only supervision.

NoiseTrue-Adaptive — End-to-End Pipeline

Single forward pass: joint denoise + deblur + 2x super-resolution, conditioned on per-image degradation severity

--- adaptive conditioning path (our novelty)

--- supervision signal, training only — not used at inference





Innovation and Uniqueness



Most submissions will train one fixed restoration network. Ours estimates degradation severity per image and adapts to it - and we proved it works.

KEY INNOVATION

Degradation-aware FiLM conditioning. A small estimator network reads each degraded input and predicts how much damage it carries; a FiLM layer converts that into a per-channel scale and shift at the bottleneck. The network therefore processes a lightly-degraded image differently from a heavily-degraded one, instead of averaging one strategy over both.

The FiLM layer is zero-initialised, so training starts exactly equivalent to the plain baseline and can only deviate if doing so measurably lowers the loss. Our first attempt used default random init and lost to the baseline on every metric; diagnosing that and fixing the initialisation is what turned the idea into a real gain.

COMPETITIVE ADVANTAGE

Validated, not assumed. We trained five models and ran a controlled ablation: U-Net baseline, U-Net + FiLM, NAFNet-lite baseline, NAFNet-lite + FiLM, and the final wide NAFNet + FiLM + VGG perceptual loss. FiLM improved results on both backbones independently, which is far stronger evidence than a single run.

Perceptual loss where it counts. Adding a VGG16 feature loss targets LPIPS directly and produced our largest single gain (0.2326 -> 0.1938). Speed was never traded away: 6.50 ms per image end-to-end including disk I/O, on a T4 - and the H100 the organisers benchmark on is materially faster still.



Results

Ablation study - 320 held-out validation pairs

Model	PSNR	SSIM	LPIPS	Params
U-Net baseline	28.236	0.7649	0.2572	1.96 M
U-Net + FILM	28.411	0.7662	0.2670	1.99 M
NAFNet-lite baseline	28.205	0.7651	0.2354	1.21 M
NAFNet-lite + FILM	28.332	0.7666	0.2326	1.23 M
FINAL: wide NAFNet+FILM+VGG	28.546	0.7682	0.1938	2.73 M

Higher PSNR / SSIM is better, lower LPIPS is better.
Final model wins on all three official metrics. Largest gain is LPIPS
(17% better than next best) - consistent with adding a VGG perceptual
loss that directly targets perceptual quality.

28.546 dB

PSNR

best of all five models tested

0.1938

LPIPS

17% better than the next-best model

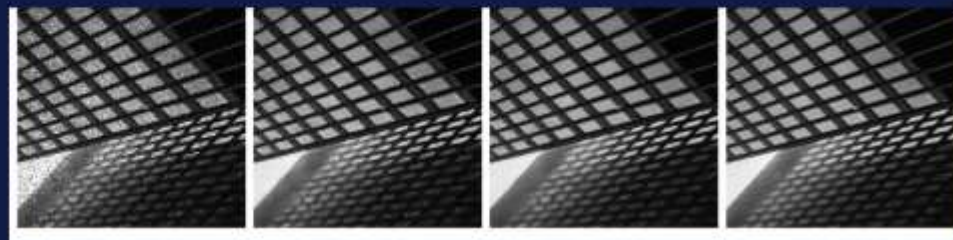
6.50 ms

per image

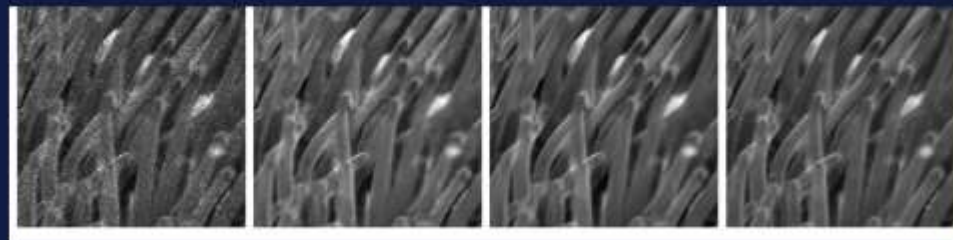
end-to-end on T4 GPU across 3200 images,
including disk I/O and pre/post-processing

Visual results

Noisy input | U-Net + FILM | Final model | Ground truth



Dense periodic grid - final model recovers lattice lines the U-Net smears



Fine fibrous texture - sharper edges, better contrast retention



Smooth gradient + lettering - no ringing or hallucinated texture



Technology & Feasibility/Methodology Used



PyTorch throughout. Trained on a single free-tier Colab T4; the whole five-model programme fits in a few GPU-hours, so every result here is reproducible on hardware any student can access.

IMPLEMENTATION STRATEGY

Training: PyTorch, Adam (lr 1e-3), cosine annealing, batch 16, 80 epochs, flip augmentation, seed 42 for a reproducible split.

Losses: Charbonnier + 0.1 x Sobel edge + 0.05 x VGG16 perceptual. Model: 2.73 M params, kept small because throughput is scored.

Inference: inference.py takes --input_dir / --output_dir, batches same-shape images, auto-detects GPU, clips output to [0, 1].

Metrics: PSNR/SSIM via scikit-image, LPIPS via the standard lpips package - reference implementations, not hand-rolled.



PyTorch . NumPy . scikit-image



Colab T4 train . H100 eval



Git . VS Code . Colab



GitHub & Video Link



GitHub Repository

<https://github.com/dharshanrajurd/NoiseTrue-Adaptive>



Prototype / Simulation Video

https://youtu.be/GsRu_q6-TAo



Research and References



Research Background & Methodology

Every architectural and loss choice below is taken from published restoration literature rather than invented for this submission.

NAFNet supplies the backbone block: SimpleGate replaces nonlinear activations, and simplified channel attention replaces heavier attention, giving a better quality-per-FLOP trade-off than a plain U-Net. FiLM supplies the conditioning mechanism - a feature-wise affine modulation driven by a side network. Charbonnier is a differentiable robust L1 variant, appropriate here because speckle produces genuine outlier pixels. The VGG perceptual loss follows the standard feature-space formulation, and LPIPS(our evaluation metric) is built on the same principle, which is why optimising the former moves the latter.



References & Citations

Papers, libraries and pretrained weights used. All licences permit competition use.

Chen, L. et al. (2022). Simple Baselines for Image Restoration (NAFNet). ECCV 2022. arxiv.org/abs/2204.04676

Perez, E. et al. (2018). FiLM: Visual Reasoning with a General Conditioning Layer. AAAI 2018. arxiv.org/abs/1709.07871

Zhang, R. et al. (2018). The Unreasonable Effectiveness of Deep Features as a Perceptual Metric (LPIPS). CVPR 2018. arxiv.org/abs/1801.03924

Simonyan, K. & Zisserman, A. (2015). Very Deep Convolutional Networks for Large-Scale Image Recognition (VGG16). ICLR 2015. arxiv.org/abs/1409.1556
Dataset: KLA paired GT / NoisyLR training set, SEMICON India Hackathon 2026. Pretrained VGG16 weights via torchvision (BSD-3); lpips package (BSD-2).